



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE3101 Bases de Datos

Documentación Técnica

Proyecto TECAir

Integrantes:

Luis Eladio Castro — 2013083843

Gabriel Soto López — 2024178797

Ariel Saborio Alvarez — 2024101248

Sofia Xie Xie — 2024100513

Profesor: Marco Rivera Meneses

Primer Semestre 2026

26 de mayo, 2026

Tabla de contenido

1.	Introducción	3
2.	Modelo conceptual.....	4
3.	Modelo relacional.....	5
3.1	Mapeo de tipo de entidades fuertes	5
3.2	Mapeo de tipo de entidades débiles	5
3.3	Mapeo de tipo de asociación 1:1.....	6
3.4	Mapeo de tipo de asociación 1:N	6
3.5	Mapeo de tipo de asociación N:M	7
3.6	Mapeo de atributos multivaluados.....	7
3.7	Modelo resultante del proceso.....	8
	 ¡Error! Marcador no definido.	
4.	Descripción de las estructuras de datos desarrolladas.....	9
4.1	Modelos	9
4.2	Persistencia	9
4.3	Interfaces de Repositorio	10
4.4	Infraestructura	10
4.5	DTOs (Data Transfer Objects)	10
5.	Descripción de la arquitectura desarrollada.	10
6.	Diagrama de Clase	14
7.	Problemas conocidos.....	15
8.	Problemas encontrados.....	16
9.	Conclusiones.....	17
10.	Recomendaciones	18
11.	Bibliografía consultada	19

1. Introducción

El presente documento describe el desarrollo técnico del sistema de gestión para la aerolínea TECAir, una empresa de transporte aéreo de bajo costo orientada principalmente a brindar soluciones accesibles para estudiantes que realizan intercambios académicos y becas internacionales.

El sistema permitirá administrar las operaciones principales de la aerolínea a través de tres plataformas interconectadas: una Vista de Reservaciones orientada a clientes, una Vista Aeropuerto destinada al personal administrativo y operativo, y una Aplicación Móvil que ofrecerá funcionalidades de reservación y compra de boletos aun sin conexión a internet.

La solución tecnológica contempla el uso de una arquitectura multicapa compuesta por una base de datos en PostgreSQL, servicios desarrollados en C#, aplicaciones web construidas con tecnologías modernas React, además de herramientas complementarias como Bootstrap, HTML5 y CSS para el diseño de interfaces responsivas y amigables. Asimismo, la aplicación móvil incorporará una base de datos local en SQLite, permitiendo la sincronización de información con el servidor principal cuando exista conectividad.

Entre las funcionalidades principales del sistema se incluyen la gestión de usuarios, búsqueda y reservación de vuelos, administración de promociones, chequeo de pasajeros, asignación de asientos y maletas, apertura y cierre de vuelos, así como la generación e impresión de pases de abordar. Además, el sistema deberá cumplir con diversos requerimientos no funcionales relacionados con rendimiento, escalabilidad, seguridad y despliegue local sobre IIS.

Este documento tiene como propósito presentar la estructura técnica del proyecto, incluyendo el análisis de requerimientos, diseño de base de datos, arquitectura del sistema, componentes de software, tecnologías utilizadas y lineamientos de implementación necesarios para garantizar el correcto funcionamiento de la solución propuesta.

2. Modelo conceptual

A continuación, se presenta el modelo conceptual del sistema TECAir, representado mediante la notación de Chen. Este tiene como objetivo describir de forma gráfica y abstracta las principales entidades del dominio, junto con sus atributos y las relaciones existentes entre ellas. Este modelo permite comprender el sistema en estructura lógica de la información y en cómo interactúan los elementos principales del negocio.

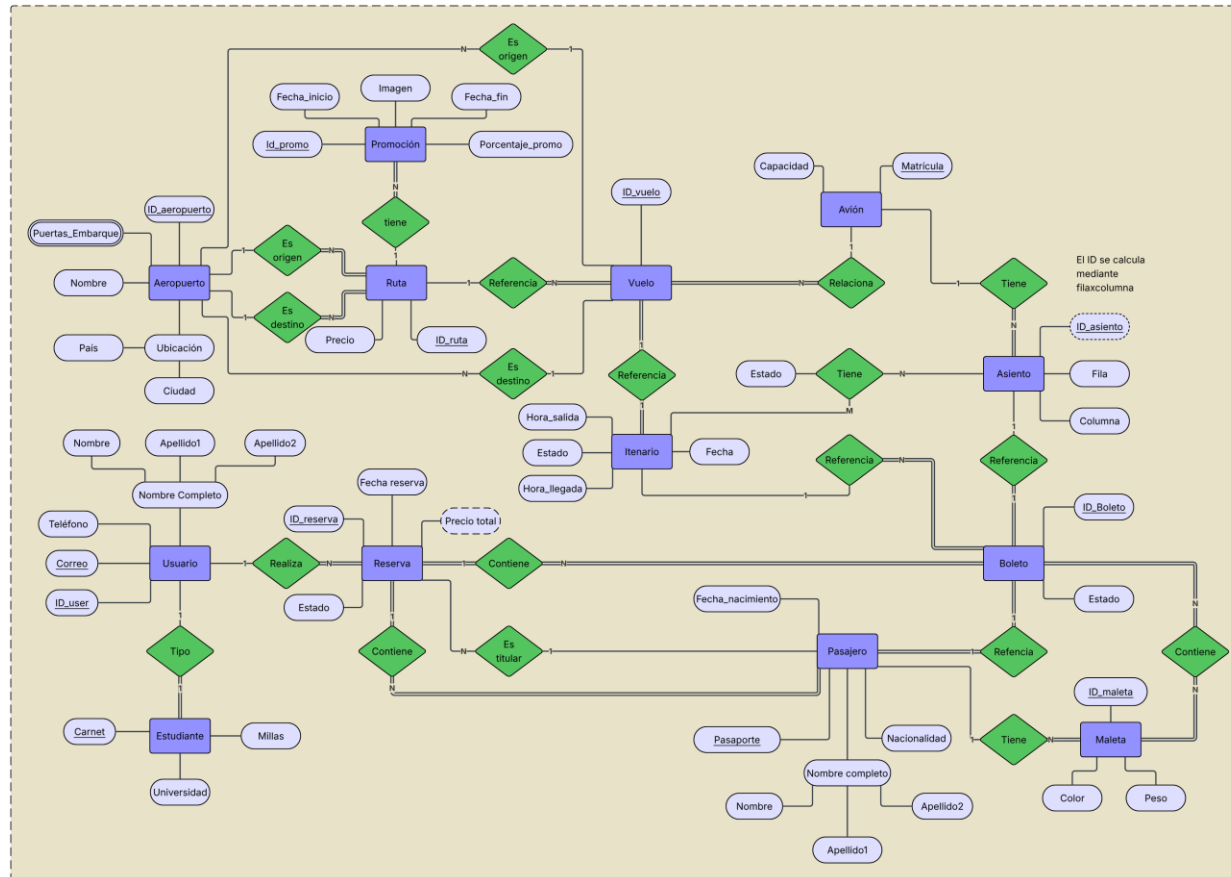


Figura.1 Modelo Conceptual TECAir

3. Modelo relacional

El modelo relacional representa la transformación del modelo conceptual a una estructura compatible con sistemas de bases de datos. Su propósito es organizar la información de manera eficiente, asegurando la integridad, consistencia y facilidad de acceso a los datos dentro del sistema TECAir. A continuación, se realiza la descripción detallada del mapeo de modelo conceptual al modelo relacional.

3.1 Mapeo de tipo de entidades fuertes

Se agregaron una relación para cada entidad fuerte del modelo conceptual, con sus respectivos atributos. Para los atributos compuestos como nombre completo en la entidad cliente y ubicación del aeropuerto se insertó atributos adicionales para almacenar cada valor individualmente. Finalmente se subraya la llave primaria de cada entidad.

Estudiante

Universidad	<u>Carnet</u>	Millas
-------------	---------------	--------

Usuario

Correo	<u>id_user</u>	Nombre	Ap1	Ap2	Teléfono
--------	----------------	--------	-----	-----	----------

Reserva

Estado	Fecha	<u>id_reserva</u>
--------	-------	-------------------

Pasajero

Nacionalidad	<u>Pasaporte</u>	Fecha_nacimiento	Nombre	Ap1	Ap2
--------------	------------------	------------------	--------	-----	-----

Maleta

<u>id_maleta</u>	Color	Peso
------------------	-------	------

Boleto

<u>id_boleto</u>	Estado
------------------	--------

Asiento

Fila	Columna	<u>id_asiento</u>
------	---------	-------------------

Aeropuerto

Nombre	País	<u>Código</u>	Ciudad	<u>id_aeropuerto</u>
--------	------	---------------	--------	----------------------

Ruta

Precio	<u>id_ruta</u>
--------	----------------

Promoción

Porcentaje	<u>id_promo</u>	Inicio	Fin	Imagen
------------	-----------------	--------	-----	--------

Vuelo

<u>id_vuelo</u>

Itinerario

Fecha	<u>id_itinerario</u>	Llegada	Salida	Estado
-------	----------------------	---------	--------	--------

Avión

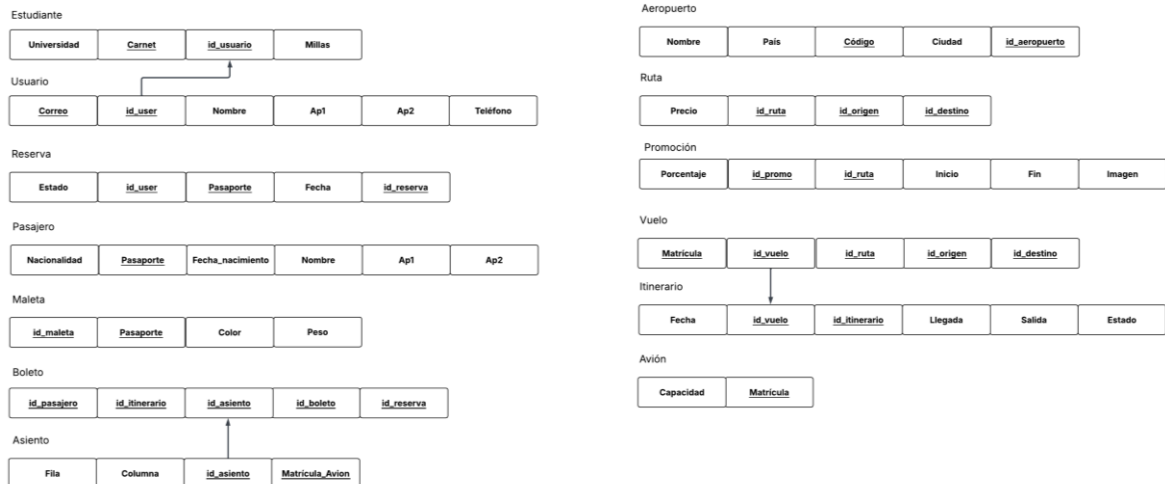
Capacidad	<u>Matricula</u>
-----------	------------------

3.2 Mapeo de tipo de entidades débiles

Para el mapeo de las entidades débiles se debe de construir una relación con sus propios atributos. Sin embargo, en este proyecto no se presentó entidades débiles.

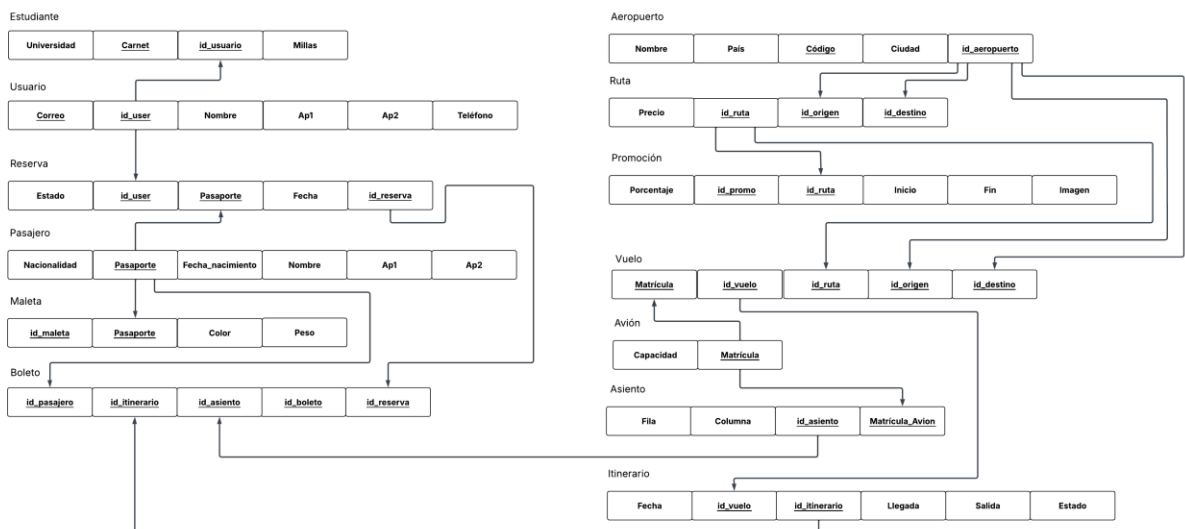
3.3 Mapeo de tipo de asociación 1:1

Se identificó las identidades que tenían una relación binaria, y después se conectó mediante una llave foránea las llaves de cada identidad. Para una entidad que tiene participación obligatoria y la otra opcional, la llave primaria de la entidad opcional debe insertarse como llave foránea en la tabla con participación total. Por ejemplo, estudiante y usuario, se inserta como llave foránea el id_usuario dentro de estudiante.



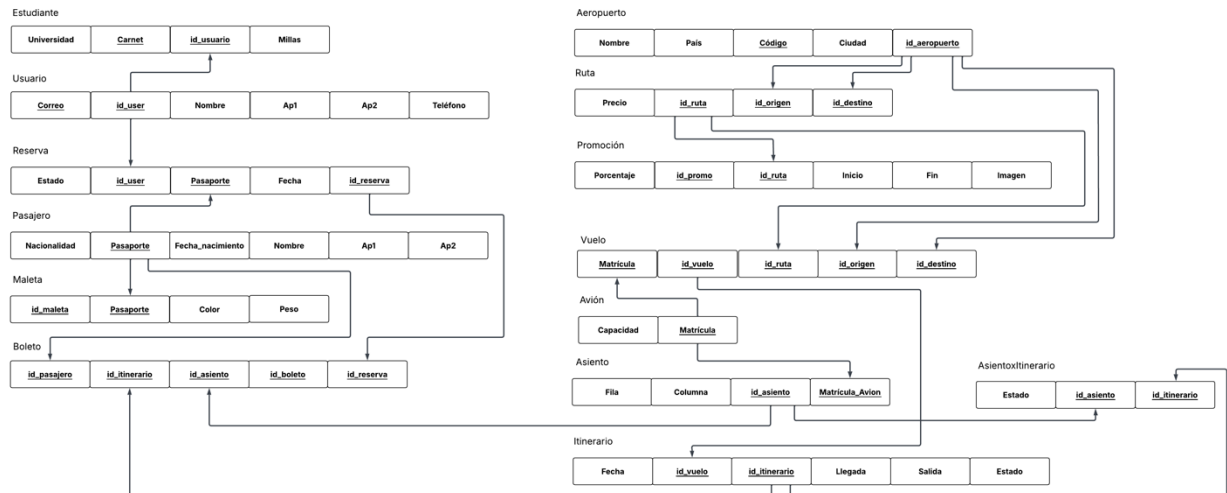
3.4 Mapeo de tipo de asociación 1:N

Se identifican las relaciones que son del tipo 1 a N. Se toma la llave primaria de la entidad padre (1) y se agrega como llave foránea a la entidad hijo (N).



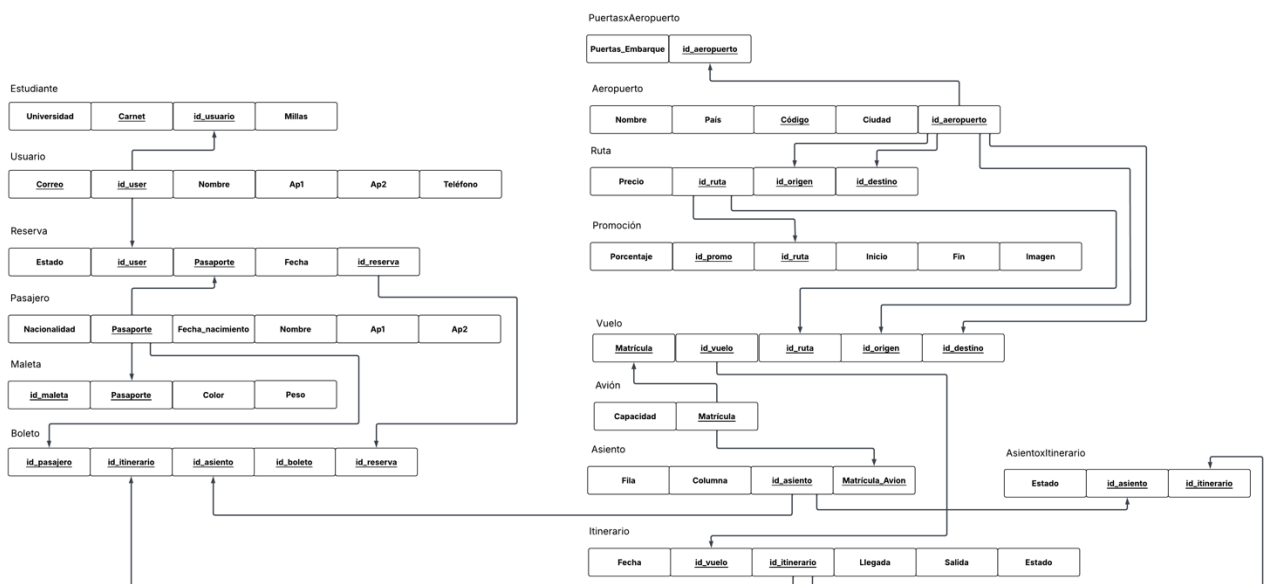
3.5 Mapeo de tipo de asociación N:M

En una relación de forma N:M, en este caso itinerario - asientos, se crea una nueva relación intermedia. De este modo la llave primaria de la nueva relación se compone de la llave primaria de ambas relaciones asociadas.



3.6 Mapeo de atributos multivaluados

Para los atributos multivaluados, tal y como las puertas de embarque dentro de los aeropuertos, se crea una relación nueva que tiene una llave formada por la llave primaria de la relación con la que se asocia el atributo multivaluado



3.7 Modelo resultante del proceso

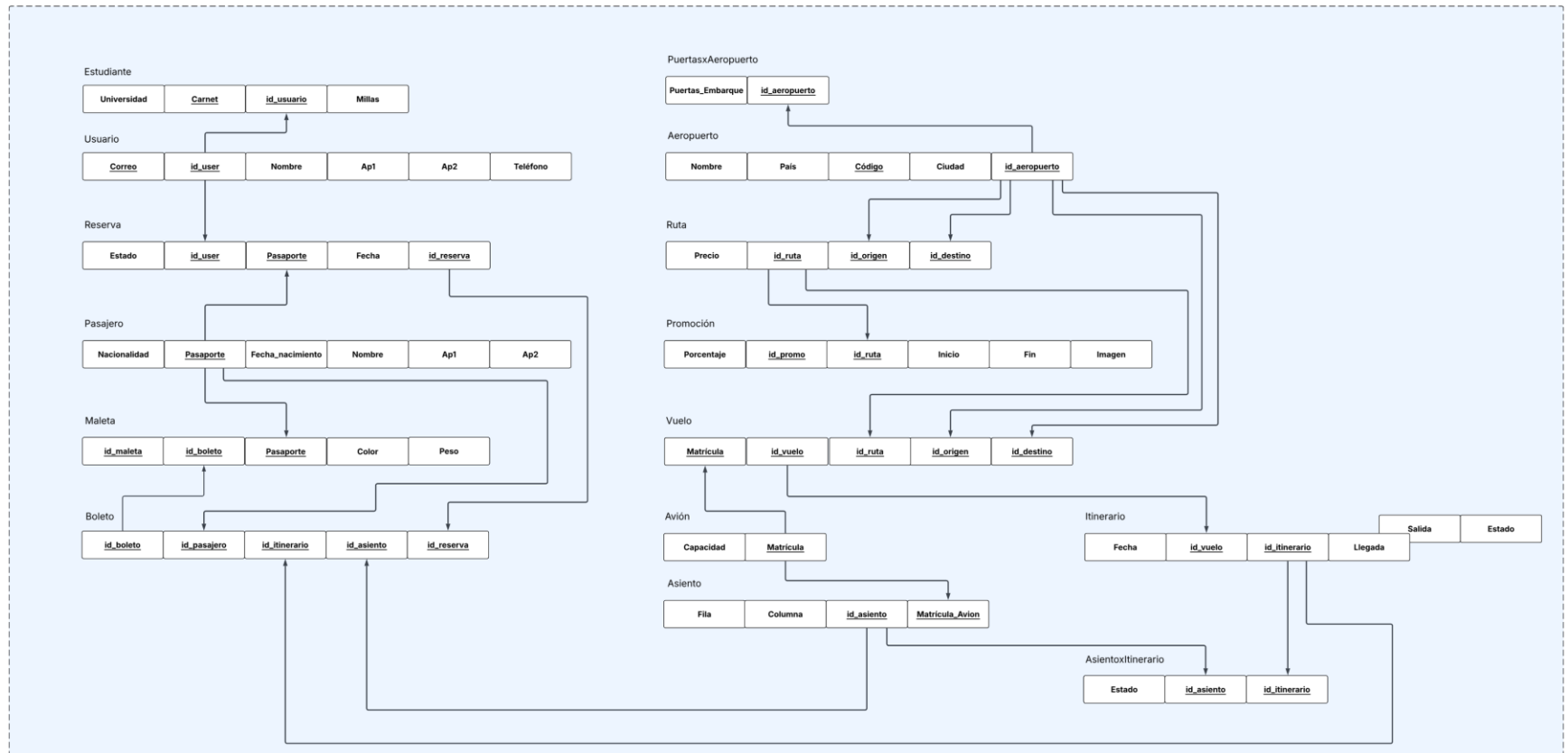


Figura.2 Modelo Relacional TECAir

4. Descripción de las estructuras de datos desarrolladas

En este apartado se describen las estructuras de datos implementadas en el sistema para el almacenamiento, administración y manipulación de la información. Para este proyecto, el almacenamiento de los datos se realiza mediante una base de datos en PostgreSQL para el sistema central, mientras que la aplicación móvil utiliza una base de datos local en SQLite que permite operar sin conexión y sincronizar la información posteriormente con el servidor principal.

4.1 Modelos

Los modelos del sistema representan una copia directa del modelo relacional diseñado para la solución del sistema. Cada clase modelo corresponde a una entidad del modelo relacional, donde sus atributos representan los campos definidos en la base de datos. Las llaves primarias y foráneas se mantienen como atributos dentro de estas clases, permitiendo representar las relaciones entre entidades tal como están definidas en el modelo. Las clases modelo son: Usuario, Estudiante, Aeropuerto, Ruta, Vuelo, Avión, Reserva, Boleto, Promoción, Pasajero, Maleta, Asiento.

4.2 Persistencia de dato con PostgreSQL y SQLite

La persistencia principal del sistema se realiza mediante PostgreSQL, donde cada entidad del modelo relacional corresponde a una tabla dentro de la base de datos. Las operaciones de inserción, actualización, eliminación y consulta se ejecutan a través de la capa de repositorios, permitiendo mantener una separación adecuada entre la lógica de negocio y el acceso a datos. Las relaciones entre entidades se implementan utilizando llaves primarias y foráneas para garantizar la integridad y consistencia de la información.

Por otra parte, la aplicación móvil incorpora una base de datos local en SQLite para permitir el funcionamiento sin conexión a internet. La información almacenada localmente incluye datos de usuarios, reservaciones y procesos pendientes de sincronización. Cuando el dispositivo recupera conectividad, la aplicación sincroniza automáticamente los cambios con la base de datos principal mediante el API centralizado.

4.3 Interfaces de Repositorio

Para desacoplar la capa de acceso a datos de la lógica de negocio, se definieron interfaces de repositorio para cada entidad del sistema. Estas interfaces definen las operaciones disponibles para cada entidad del sistema, incluyendo operaciones básicas de consulta, inserción, actualización y eliminación (CRUD).

Las implementaciones concretas de estas interfaces son las encargadas de interactuar directamente con PostgreSQL mediante consultas SQL. Esta separación permite mantener una arquitectura limpia y facilita futuras modificaciones tecnológicas sin afectar la lógica de negocio ni los controladores de la aplicación.

4.4 Infraestructura

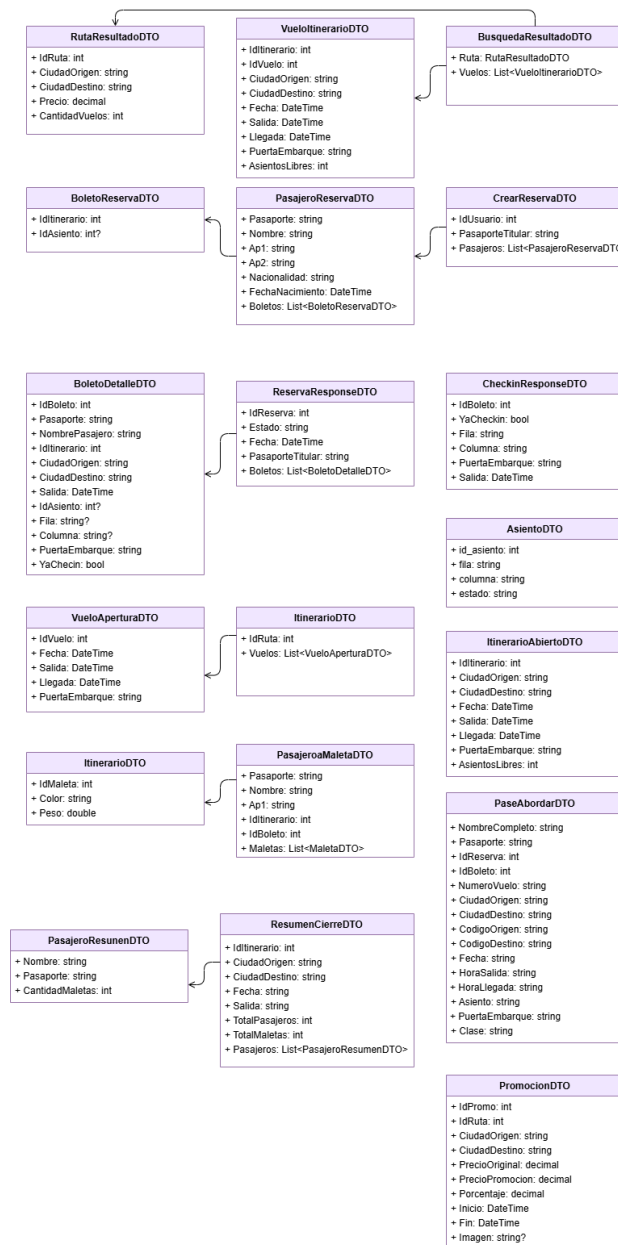
Para gestionar la persistencia y el acceso a los datos, el sistema implementa una infraestructura basada en una capa de repositorios y conexiones administradas hacia PostgreSQL. Esta capa encapsula las operaciones de lectura, escritura, actualización y eliminación de información, permitiendo desacoplar el acceso a datos de la lógica de negocio. Además, se utilizan mecanismos de conexión centralizados que optimizan la comunicación con la base de datos y facilitan el mantenimiento de la aplicación.

Por otra parte, la aplicación móvil incorpora una base de datos local en SQLite que permite operar sin conexión a internet. La información almacenada localmente se sincroniza posteriormente con la base de datos principal mediante el API centralizado cuando el dispositivo recupera conectividad. Esta arquitectura permite mantener disponibilidad y consistencia de la información entre las distintas plataformas del sistema.

4.5 DTOs (Data Transfer Objects)

Los DTOs son objetos utilizados para transferir información entre las distintas capas del sistema cuando la información requerida no corresponde directamente a una única entidad del modelo, sino a una combinación de varias entidades relacionadas. A diferencia de los modelos, los DTOs no representan tablas de la base de datos, sino estructuras diseñadas específicamente para el intercambio de información entre el backend, el frontend y la aplicación móvil.

El sistema cuenta con DTOs para representar información consolidada de vuelos, reservaciones, promociones, check-ins, equipaje y pases de abordar. Estos objetos permiten agrupar información proveniente de múltiples entidades y construir respuestas optimizadas para cada funcionalidad del sistema, reduciendo el acoplamiento entre capas y mejorando la eficiencia en la transferencia de datos.



5. Descripción de la arquitectura desarrollada.

La arquitectura del sistema TECAir se basa en un enfoque multicapa orientado a servicios, el cual permite separar claramente la interfaz de usuario, la lógica de negocio y el acceso a datos. Esta organización facilita la escalabilidad, el mantenimiento y la modularidad del sistema, permitiendo que cada componente tenga responsabilidades bien definidas. El frontend está desarrollado utilizando tecnologías web modernas como React, mientras que la capa de servicios está implementada en C# con .NET, encargándose de procesar la lógica del sistema y comunicarse con las bases de datos del proyecto.

5.1 Capas del sistema

La arquitectura está compuesta por una capa de presentación, una capa de controladores, una capa de servicios y una capa de acceso a datos. Cada una de estas capas cumple una función específica dentro del sistema y se comunica con la siguiente de forma controlada y unidireccional, garantizando una adecuada separación de responsabilidades.

La capa de presentación corresponde a las aplicaciones web y móvil desarrollado en React, las cuales permiten al usuario interactuar con el sistema. A través de esta capa se presenta la interfaz de usuario tanto para la vista cliente como para la vista administrador, dentro de ello realizan procesos como la búsqueda de vuelos, la gestión de reservaciones, el check-in de pasajeros, la consulta de promociones y la administración del equipaje. La comunicación con el backend se realiza mediante solicitudes HTTP utilizando el protocolo REST, lo que permite la integración entre todas las plataformas del sistema.

La capa de controladores corresponde a la API REST, desarrollada en C# con .NET 9 y desplegada en Internet Information Services (IIS). Esta capa expone los endpoints del sistema y actúa como punto central de comunicación entre las aplicaciones cliente y la lógica de negocio. Su función principal es recibir las solicitudes HTTP provenientes del frontend y de la aplicación móvil, procesar las validaciones necesarias y delegar las

operaciones correspondientes hacia la capa de servicios, retornando posteriormente las respuestas serializadas en formato JSON.

La capa de servicios concentra toda la lógica de negocio del sistema TECAir. En esta capa se implementan las reglas y operaciones propias de un conjunto de entidades relacionadas. Además, esta capa se encarga de coordinar la interacción con la capa de repositorios para obtener y persistir datos, y construyen los DTOs necesarios para retornar vistas compuestas al frontend.

Finalmente, la capa de repositorios es responsable del acceso y persistencia de los datos del sistema. Cada repositorio implementa una interfaz que define las operaciones disponibles sobre una entidad específica, permitiendo desacoplar completamente la lógica de negocio del mecanismo de persistencia. Esta separación facilita el mantenimiento, reutilización y escalabilidad de la aplicación.

La infraestructura de acceso a datos se basa en conexiones administradas hacia PostgreSQL, utilizando consultas SQL desarrolladas en C# para ejecutar operaciones de inserción, consulta, actualización y eliminación de información. Los repositorios encapsulan estas operaciones y centralizan la interacción con la base de datos, garantizando consistencia en el acceso a la información y evitando dependencias directas entre la lógica de negocio y la capa de persistencia.

Adicionalmente, la aplicación móvil incorpora una base de datos local en SQLite para permitir el funcionamiento sin conexión a internet. Cuando el dispositivo no dispone de conectividad, la información se almacena temporalmente de forma local y posteriormente se sincroniza con PostgreSQL mediante la API REST del sistema una vez restablecida la conexión.

5.2 Flujo de comunicación

El flujo de comunicación entre capas del sistema sigue una estructura bien definida, en la cual la información viaja desde la capa de presentación como una solicitud HTTP hacia los controladores correspondientes. Posteriormente, los controladores invocan a la

capa de servicios responsable de la operación, donde se aplica la lógica de negocio y se procesan las validaciones necesarias del sistema.

Una vez ejecutada la lógica correspondiente, la capa de servicios consulta o actualiza la información a través de los repositorios, los cuales gestionan la comunicación con la base de datos PostgreSQL mediante consultas SQL desarrolladas en C#.

En el caso de la aplicación móvil, cuando no existe conexión a internet, las operaciones se almacenan temporalmente en SQLite y posteriormente son sincronizadas con PostgreSQL a través de la API REST una vez restablecida la conectividad. Este flujo permite mantener consistencia, control y organización en la comunicación entre las distintas capas del sistema.

5.3 Patrones de diseño

Uno de los principales patrones que emplea el sistema es el MVC, la cual separa las responsabilidades entre la vista, los controladores, los servicios con la lógica de negocio y los modelos de datos. También se implementa el patrón Repositorio la cual abstrae el acceso a datos mediante interfaces que definen las operaciones CRUD de cada entidad, permitiendo que la lógica de negocio sea independiente del mecanismo de persistencia utilizado, ya que los servicios no interactúan directamente con los datos, sino a través de los repositorios que encapsulan las consultas SQL hacia PostgreSQL. Otro patrón implementado es el uso de DTOs, para la transferencia de información en distintas capas del sistema. Además, se emplea la inyección de dependencias, para la creación y gestión de instancias de servicios y repositorios. Finalmente, se aplica el principio de separación de responsabilidades dentro de la arquitectura, para garantizar que cada componente del sistema tenga una única función definida dentro del flujo general de la aplicación.

6. Diagrama de Clase

El diagrama de clases del sistema TECAir representa la estructura lógica del sistema a nivel de diseño, mostrando las principales entidades que componen el dominio y la forma en que estas se relacionan entre sí. Este diagrama permite visualizar la organización del sistema desde una perspectiva orientada a objetos, facilitando la comprensión de la interacción entre las diferentes clases que conforman la aplicación.

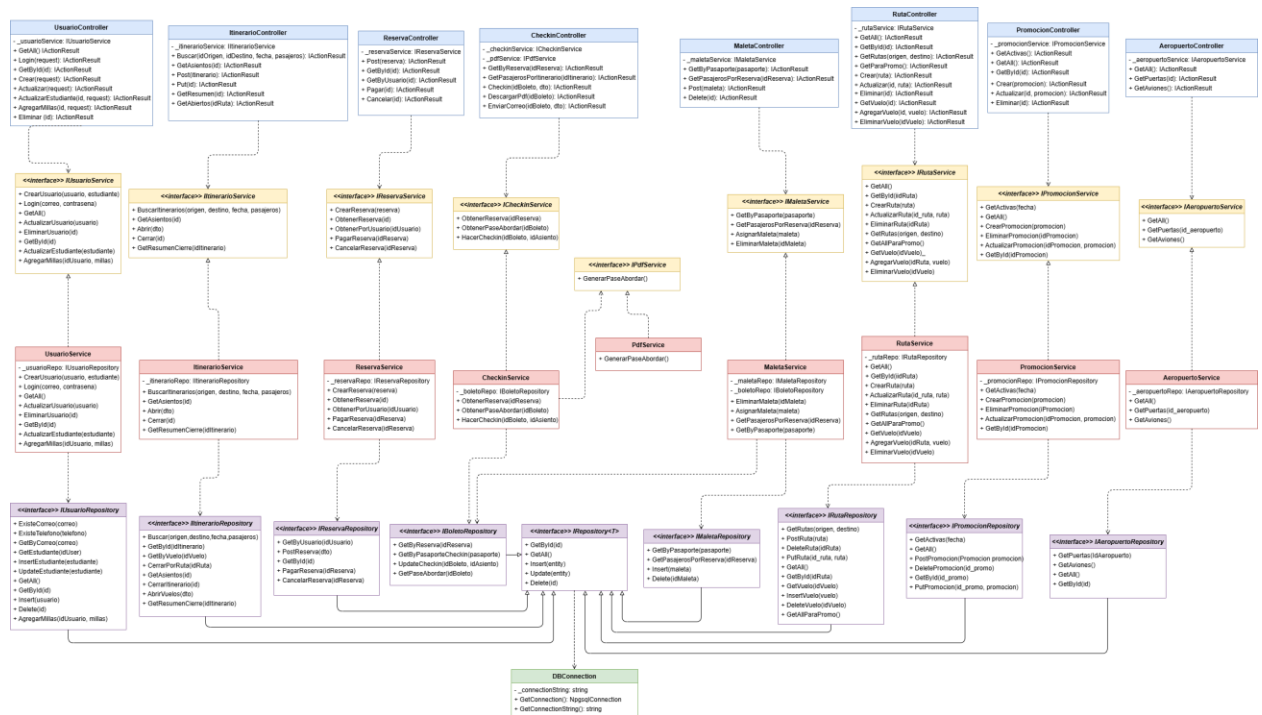


Figura 3. Diagrama de Clases

7. Problemas conocidos

En el frontend del cliente, no existe validación de edad mínima para los pasajeros al momento de registrar sus datos en el formulario de reserva. Tampoco se valida la longitud máxima de los campos de texto como nombre, apellidos y número de pasaporte.

En el frontend del administrador, el formulario de rutas no valida que el aeropuerto de origen y el de destino sean diferentes. En la apertura de vuelos no se valida que la hora de llegada sea posterior a la hora de salida, ni que la fecha del vuelo sea una fecha futura. Tampoco se valida si un avión esta ocupado a la hora de asignarlo a un vuelo.

En el backend, no existe validación que impida abrir un itinerario duplicado para el mismo vuelo y fecha. Tampoco hay validación de edad mínima para volar a nivel de servicio.

Adicionalmente, el cierre automático de vuelos una hora antes de la salida está pendiente de implementar actualmente el cierre es únicamente manual desde el panel de administración.

8. Problemas encontrados

Durante el desarrollo del sistema TECAir se presentaron diversos retos técnicos y organizativos asociados a la integración de múltiples tecnologías, la gestión de datos distribuidos y la implementación de reglas de negocio complejas dentro del sistema.

Uno de los principales problemas encontrados fue la instalación y configuración de SQLite en el entorno de desarrollo de la aplicación móvil. Durante este proceso se presentaron dificultades relacionadas con la integración de la base de datos local con el proyecto, así como la correcta configuración de las dependencias necesarias para su funcionamiento. Inicialmente, la base de datos no lograba persistir correctamente los datos en el dispositivo, lo que afectaba el funcionamiento offline de la aplicación. La solución consistió en revisar la configuración del entorno, asegurar la correcta instalación del paquete de SQLite y validar la creación de la base de datos local al momento de la ejecución de la aplicación, garantizando así su funcionamiento adecuado sin conexión a Internet.

Del mismo modo, se enfrentó problemas con la sincronización entre la aplicación móvil y la base de datos principal cuando el dispositivo operaba sin conexión. Inicialmente, se presentaron inconsistencias al momento de sincronizar la información almacenada localmente en SQLite con PostgreSQL, lo que provocaba duplicación de registros o pérdida parcial de información. Como intento de solución, los datos almacenados localmente se sincronizaron con la base de datos central mediante la API REST. Este proceso incluye validaciones en la capa de servicios para garantizar la integridad de los datos antes de su persistencia en PostgreSQL.

Otro desafío importante fue el manejo de relaciones complejas dentro de la base de datos, especialmente en los módulos de reservaciones, vuelos, check-in y asignación de equipaje. La dificultad principal radicó en modelar correctamente las relaciones entre entidades, debido a que un vuelo puede contener múltiples pasajeros y cada pasajero puede registrar varias maletas con reglas de cobro diferenciadas. Esto generó inicialmente inconsistencias en los cálculos de tarifas adicionales y en la asignación de estados durante el proceso de check-in. La solución se logró mediante la centralización

de las reglas de negocio en la capa de servicios y la implementación de validaciones previas a la persistencia de datos.

Asimismo, se presentaron dificultades en el almacenamiento y manejo de imágenes dentro del sistema, específicamente en la gestión de promociones. Inicialmente, se intentó almacenar las imágenes directamente en la base de datos, lo cual generó problemas de rendimiento y complejidad en la recuperación de la información. Posteriormente, se optó por almacenar únicamente la referencia o ruta de la imagen, lo que permitió optimizar el rendimiento y simplificar el acceso desde la aplicación.

En la capa de frontend también se presentaron conflictos de estilos debido a la combinación de CSS personalizado con Bootstrap. Esto provocó inconsistencias visuales en algunos componentes de la interfaz, como solapamiento de estilos y comportamiento no uniforme en diferentes vistas. Como solución, se realizó una reorganización de las clases CSS, priorizando el uso de Bootstrap para la estructura general y reservando estilos personalizados únicamente para ajustes específicos, lo que permitió estabilizar la apariencia del sistema.

9. Conclusiones

El desarrollo del sistema TECAir permitió aplicar de manera práctica los conceptos relacionados con arquitectura de software, bases de datos relacionales, desarrollo web y aplicaciones móviles. A través de la implementación del proyecto se logró construir una solución capaz de centralizar los principales procesos operativos y administrativos de una aerolínea, integrando funcionalidades de reservación, gestión de vuelos, check-in, promociones y administración de equipaje.

La utilización de una arquitectura multicapa permitió mantener una adecuada separación de responsabilidades entre la interfaz de usuario, la lógica de negocio y el acceso a datos, facilitando la organización y mantenibilidad del sistema. Asimismo, el uso de PostgreSQL como base de datos principal garantizó integridad y consistencia en la información gestionada por la aplicación.

La implementación de una aplicación móvil con SQLite permitió incorporar soporte para funcionamiento offline, mejorando la disponibilidad del sistema y permitiendo la

sincronización de información cuando existe conectividad. Esto representó una solución eficiente para escenarios donde el acceso a internet puede ser limitado.

Además, el uso de tecnologías modernas como C#, .NET, React y Bootstrap permitió desarrollar interfaces dinámicas y una API REST centralizada que facilita la comunicación entre las distintas plataformas del sistema.

Finalmente, el proyecto permitió fortalecer conocimientos relacionados con patrones de diseño, desarrollo por capas, repositorios, DTOs y manejo de servicios REST, evidenciando la importancia de una arquitectura bien estructurada para proyectos de software escalables y mantenibles.

10. Recomendaciones

Para futuras versiones del proyecto, podría considerarse la implementación de procesos automáticos de sincronización más robustos en la aplicación móvil, optimizando el manejo de conflictos entre datos locales y remotos.

Se recomienda continuar fortaleciendo la arquitectura del sistema mediante la implementación de mejoras en la capa de sincronización entre la aplicación móvil y la base de datos principal, con el objetivo de manejar de forma más eficiente escenarios de pérdida de conexión y conflictos de datos durante la sincronización.

Asimismo, sería conveniente agregar pruebas automatizadas tanto para la API como para la interfaz de usuario, con el fin de mejorar la calidad del software y reducir errores durante futuras modificaciones.

También se sugiere optimizar el manejo de archivos multimedia dentro del sistema, especialmente en la gestión de imágenes asociadas a promociones. En lugar de almacenar rutas locales, se recomienda considerar el uso de servicios de almacenamiento en la nube o sistemas especializados para la gestión de archivos, lo que permitiría mejorar la escalabilidad y disponibilidad de los recursos.

En cuanto a la base de datos, se recomienda implementar estrategias adicionales de validación y optimización de consultas SQL en PostgreSQL, con el fin de mejorar el rendimiento del sistema en escenarios de alta carga de usuarios y grandes volúmenes de datos.

En el área del frontend, se recomienda establecer una estandarización más estricta en el uso de frameworks de estilos como Bootstrap y CSS personalizado, con el fin de evitar conflictos visuales y garantizar una interfaz más consistente en todas las vistas del sistema.

Finalmente, se recomienda mantener una adecuada documentación del código y de la arquitectura del sistema, así como el uso de buenas prácticas de desarrollo como la separación de responsabilidades, el uso de patrones de diseño y la aplicación de pruebas en la capa de servicios, lo cual facilitará futuras mejoras y mantenimiento del sistema.

11. Bibliografía consultada

Rodríguez, F. (2022). Cómo navegar programáticamente entre páginas usando React Router [Video]. YouTube. <https://www.youtube.com/watch?v=8WCoWFkzHkA&list=LL&index=2>

GeeksforGeeks. (2021, Octubre 21). What are the features of ReactJS ? GeeksforGeeks. <https://www.geeksforgeeks.org/reactjs/what-are-the-features-of-reactjs/>

Meta Platforms, Inc. (2026). React documentation. <https://react.dev/>

leanser. (2026, Marzo 27). Información general sobre el servidor web de IIS. Microsoft. <https://learn.microsoft.com/es-es/iis/get-started/introduction-to-iis/iis-webserver-overview>

Microsoft. (2026). Documentación oficial de .NET. <https://learn.microsoft.com/dotnet/>

Postman, Inc. (2026). Postman API platform documentation. <https://learning.postman.com/>

Twarong, A. (2020, Agosto 8). React Bootstrap Tutorial [Video]. YouTube. <https://www.youtube.com/watch?v=8pKjULHzs0s>

ProgrammingKnowledge. (2025, Enero 20). Cómo instalar PostgreSQL en Windows 11 (2025) [Video]. Youtube. <https://www.youtube.com/watch?v=GpqJzWCcQXY>

PostgreSQL Global Development Group. (s. f.). *PostgreSQL*.
<https://www.postgresql.org/>

SQLite Consortium. (s. f.). *SQLite Home Page*. <https://www.sqlite.org/index.html>